

Práctica 3: ¿Qué habitación es esta?

Redes bayesianas para un robot doméstico

En esta práctica el robot no se mueve: se queda en una habitación e intenta averiguar de qué tipo es (cocina, dormitorio, baño o salón) a partir de los objetos que su clasificador **ruidoso** detecta (puede fallar al detectar un objeto presente, o “ver” uno que no está). Se modela como una **red bayesiana**: una variable oculta `RoomType` y una variable booleana `Detected_<objeto>` por cada tipo de objeto del catálogo de CasaRobot, y se responde con el algoritmo de **eliminación de variables**.

Se trabajará sobre el fichero `practica3_enunciado.py`, proporcionado parcialmente implementado. Se asume familiaridad con la API de `HouseWorld/Robot` (documentada en `practica0_intro.tex`); en concreto, con la tabla de afinidad objeto↔habitación (`OBJECT_CATALOG`), que es la que le da sentido estadístico a esta práctica.

1. Entrega

La entrega se realizará en el aula, durante la clase de prácticas del 27 de noviembre de 2026, a través de la tarea habilitada para ello en el aula virtual.

El día de la entrega, una vez entregada la práctica, se cumplimentará en el aula, en modo examen y sin acceso al código, un breve cuestionario en el que se pedirá explicar cómo se ha resuelto alguna cuestión concreta de la solución entregada (ver Sección 4 para un ejemplo del estilo de pregunta que cabe esperar).

2. Infraestructura proporcionada

Todo lo de esta sección ya está implementado en `practica3_enunciado.py`: no es TODO.

2.1. La clase Factor

Un factor es una tabla de probabilidad sobre un conjunto de variables.

<code>Factor(variables, domains, table)</code>	<code>variables</code> : nombres de variable, en el orden que fija cómo se leen las claves de <code>table</code> . <code>domains</code> : diccionario <i>compartido por toda la red</i> variable → sus valores posibles (no solo los de este factor). <code>table</code> : diccionario tupla-de-valores → probabilidad.
<code>factor.value(assignment)</code>	Probabilidad de este factor para una asignación completa (diccionario variable → valor).

Por ejemplo, un factor $P(\text{Detected_nevera} \mid \text{RoomType})$ tiene `variables = ('RoomType', 'Detected_nevera')` y una fila en `table` por cada combinación de (tipo de habitación, detectado o no) = $2 \times 4 = 8$ filas.

2.2. Utilidades de la red concreta

<code>room_type_variable()</code>	el nombre de variable <code>RoomType</code> (constante)
<code>detected_variable(obj)</code>	<code>Detected_</code> seguido de <code>obj</code>
<code>all_object_types()</code>	los 13 nombres de objeto distintos de <code>OBJECT_CATALOG</code> (todas las habitaciones), en orden estable
<code>default_elimination_order(vars, query, evidence)</code>	todas las variables de la red que no son ni consulta ni evidencia, en orden

Y para simular al robot detectando objetos en una `HouseWorld` real (útil para probar las Preguntas 6 y 7 con datos realistas, y lo que usa el `main()` de la demo):

<code>true_objects_in_room(world, room)</code>	qué objetos hay <i>realmente</i> en esa habitación (verdad fundamental, sin ruido)
<code>simulate_noisy_detection(world, room, object_types, rng, ...)</code>	para cada tipo de objeto, decide si se “detecta” aplicando las tasas de error del sensor a si está realmente presente

3. Desarrollo de la práctica

3.1. Paso 1: Ejecución inicial

Ejecuta el fichero tal cual se proporciona:

```
python3 practica3_enunciado.py
```

Debería generarse una casa y listar sus habitaciones (con su tipo real, que en el resto de la práctica se trata como oculto). Nada de lo que hay que implementar se llama todavía.

3.2. Paso 2: Preguntas 1 y 2 — la red bayesiana concreta

Q1.

```
construct_house_bayes_net(object_types)
-> (variables: List[str], domains: Dict[str, Tuple])
```

La topología: la variable `RoomType` (dominio los 4 tipos de habitación, `tuple(RoomType)`) y una variable booleana `Detected_<obj>` por cada nombre de `object_types`. `variables` debe listar `RoomType` primero y luego los `Detected_*` en el mismo orden que `object_types`.

Q2.

```
fill_detection_cpts(
    variables, domains, object_types, room_catalog=OBJECT_CATALOG, *,
    object_density=0.7, false_positive_rate=0.1, false_negative_rate=0.1
) -> List[Factor]
```

Devuelve la lista de factores CPT de la red: un factor $P(\text{RoomType})$ con prior **uniforme** sobre los 4 tipos, y un factor $P(\text{Detected}_X \mid \text{RoomType})$ por cada objeto X de `object_types`, con

$$P(\text{Detected}_X=T \mid \text{RoomType}=r) = \begin{cases} d(1-f_n) + (1-d)f_p & \text{si } X \in \text{room_catalog}[r] \\ f_p & \text{en caso contrario} \end{cases}$$

donde $d = \text{object_density}$, $f_p = \text{false_positive_rate}$, $f_n = \text{false_negative_rate}$. La idea: X solo puede estar *realmente* presente si pertenece al catálogo de esa habitación (así genera objetos `HouseWorld`, con probabilidad d por objeto); a partir de ahí se combina con las dos formas en que puede fallar el sensor (falso negativo si está y no se detecta, falso positivo si no está y se detecta igualmente).

3.3. Paso 3: Pregunta 3 — join_factors

```
join_factors(factors: List[Factor]) -> Factor
```

Combina varios factores en uno: las variables del resultado son la unión de las variables de entrada (sin repetir, en el orden en que aparecen); su tabla es, para cada combinación de valores de esas variables, el **producto** de los valores que tiene cada factor de entrada en la sub-asignación correspondiente a sus propias variables (`factor.value(...)`, dado).

3.4. Paso 4: Pregunta 4 — eliminate

```
eliminate(factor: Factor, variable: str) -> Factor
```

Marginaliza `variable` fuera de `factor`: para cada combinación de valores del resto de variables, **suma** la probabilidad sobre todos los valores posibles de `variable`. El resultado tiene una variable menos que `factor`.

3.5. Paso 5: Pregunta 5 — normalize

```
normalize(factor: Factor) -> Factor
```

Divide toda la tabla de `factor` por su suma total, de forma que la tabla resultante sume 1.

3.6. Paso 6: Pregunta 6 — eliminación de variables

```
inference_by_variable_elimination(  
    factors: List[Factor], query_variables: List[str],  
    evidence: Dict[str, Any], elimination_order: List[str]  
) -> Factor
```

El algoritmo general, apoyándose en las tres preguntas anteriores:

1. Restringe cada factor con `evidence` (dado, función interna `_restrict`: fija a su valor observado las variables de evidencia que aparezcan en cada factor, y las retira de su lista de variables).
2. Para cada variable de `elimination_order`, en ese orden: junta (`join_factors`) todos los factores *actuales* que la mencionan, elimina (`eliminate`) esa variable del resultado, y sustituye esos factores por el nuevo factor en la lista.
3. Junta lo que quede (debe mencionar solo `query_variables`) y normaliza (`normalize`).

El orden de eliminación no afecta al resultado (solo, potencialmente, al coste); en esta práctica se usa siempre `default_elimination_order` (dado), así que no hace falta preocuparse por elegir un buen orden.

3.7. Paso 7: Pregunta 7 — best_room_guess

```
best_room_guess(factors, evidence, elimination_order)  
    -> (RoomType, float)
```

Usa `inference_by_variable_elimination` para calcular $P(\text{RoomType} \mid \text{evidence})$ y devuelve el tipo de habitación con mayor probabilidad posterior, junto con esa probabilidad (pensada como una medida de confianza en la respuesta).

Descomenta el resto del `main` y Pruébalo sobre varias semillas: con `--fp-rate 0.1 --fn-rate 0.1` (por defecto) se debería acertar la gran mayoría de las habitaciones de una casa, no todas — el sensor es ruidoso a propósito.

4. Ejemplo ilustrativo de pregunta del cuestionario

Por ejemplo: añadir un tipo de habitación nuevo al catálogo (un “despacho”, con dos o tres objetos típicos propios) y pedir recalcular a mano, con una evidencia concreta dada, el valor de $P(\text{RoomType} \mid \text{evidence})$ tras incorporarlo — mismo patrón que el Ejercicio 1 de la modificación de diciembre de 2025 (red médica nueva + `inferenceByVariableElimination` con evidencia y orden de eliminación dados). Esto no es la pregunta real, solo un ejemplo del estilo esperado.

5. Para reflexionar (no se entrega, pero puede salir en el examen)

Como ejercicio de calentamiento opcional, prueba a construir a mano — con `Factor` directamente, sin pasar por `construct_house_bayes_net/fill_detection_cpts` — una red pequeña de 2 o 3 variables para un problema distinto (por ejemplo: “¿es seguro que la batería aguante la tarea?” en función del nivel de batería, la distancia a recorrer y el número de obstáculos) y responde una consulta con tu propio `inference_by_variable_elimination`. Sirve para comprobar que el algoritmo de la Pregunta 6 es realmente *agnóstico al dominio*: no sabe nada de habitaciones ni de objetos, solo de factores.

También puede ser útil pensar: ¿por qué el prior de `RoomType` es uniforme aquí, y no, por ejemplo, proporcional al número real de habitaciones de cada tipo en la casa? ¿Cambiaría mucho el resultado si lo fuera?

6. Entregables

- Un único fichero `practica3_enunciado.py` completado, renombrado a `practica3.py`, entregado en la tarea del aula virtual habilitada para ello.
- El código debe ser legible y estar debidamente tabulado. No es necesario comentarlo explicando su funcionalidad.